PERFORMANCE TEST PLAN

DemoBlaze Product Store

www.demoblaze.com

Document Version	1.0
Date	14 March 2026
Application Under Test	DemoBlaze — www.demoblaze.com
Performance Tool	Apache JMeter 5.6.3
Prepared By	AmaliTech Training Academy — Team 1
Test Types	Baseline, Load, Peak, Stress, Endurance
CI/CD Integration	GitHub Actions

1. Introduction

DemoBlaze is a publicly available e-commerce demo application hosted at www.demoblaze.com. It simulates a real-world online store offering electronics products including phones, laptops, and monitors. The application provides core e-commerce workflows including product browsing, product detail viewing, shopping cart management, and checkout.

The purpose of this performance test plan is to define the strategy, scope, objectives, and approach for conducting load and performance testing against the DemoBlaze application using Apache JMeter 5.6.3. The plan establishes the test scenarios, thread group configurations, performance metrics, and success criteria that will guide the testing effort.

Performance testing is critical to ensure that the application responds within acceptable time limits under varying levels of concurrent user load, that it remains stable during sustained usage, and that bottlenecks are identified and reported before the application is subjected to real-world traffic.

1.2 Objectives

The objectives below define what the performance tests are intended to prove or disprove. Each objective maps directly to a measurable metric that will be evaluated during test execution.

- Validate that response times remain within the acceptable threshold of 2 seconds for all critical user transactions, including login, product browsing, cart operations, and checkout.
- Verify that the application is capable of sustaining a throughput of at least 500 requests per second under peak load conditions.
- Confirm that the error rate does not exceed 1% of total requests under maximum load, ensuring system reliability.
- Identify any performance bottlenecks by subjecting the application to progressively increasing concurrent user loads.
- Evaluate the application's scalability by comparing key performance metrics across baseline, load, peak, and stress test levels.
- Assess the stability of the application under sustained load through endurance testing, with the aim of detecting memory leaks or gradual performance degradation over time.
- Automate performance test execution within a GitHub Actions pipeline to enable continuous performance monitoring on every code push.

1.3 Scope

The scope section draws a clear boundary around what will and will not be tested. Items listed as out of scope are excluded deliberately — not because they are unimportant, but because they fall outside the boundaries of this particular test cycle.

The following functional areas of the DemoBlaze application are included within the scope of this performance test:

- User registration and login — the sign-up and sign-in workflow using session-based authentication.
- Homepage loading and category browsing — navigation across Phones, Laptops, and Monitors categories.
- Product detail page loading — retrieval and display of individual product information.

- Add to Cart functionality — the AJAX-based API call to add a selected product to the user's cart.
- Cart view and item management — loading cart contents via the view cart API endpoint.
- Checkout and order placement — the full order submission workflow via the send order endpoint.

The following table shows all test types and their scope status:

Test Type	Definition	In Scope	Responsibility
Baseline Testing	Establishes performance benchmarks under light load (50 users)	**Yes**	QA Engineer
Load Testing	Validates performance under normal expected load (150 users)	**Yes**	QA Engineer
Peak Load Testing	Tests application under maximum expected traffic (300 users)	**Yes**	QA Engineer
Stress Testing	Pushes application beyond capacity to identify breaking point (500 users)	**Yes**	QA Engineer
Endurance Testing	Sustained load over 30 minutes to detect memory leaks and degradation	**Yes**	QA Engineer
Spike Testing	Sudden sharp increases in load	**No**	Out of scope
Security Testing	Vulnerability and penetration testing	**No**	Out of scope

11. Risks, Assumptions and Constraints

This section manages expectations by documenting the factors that may affect test execution or results. Identifying risks and assumptions upfront makes the test results more credible and easier to interpret in context.

Risks and Mitigations

Risk	Likelihood	Impact	Mitigation Strategy
DemoBlaze is a shared public demo application — other users may access it during test runs, inflating response times or causing unrelated errors.	**High**	**High**	Schedule test runs during off-peak hours; document shared environment limitation in the final report.
Rate limiting by demoblaze.com during high-load tests — throttling or blocking repeated calls from the same IP.	**Medium**	**High**	Add Gaussian think times between requests; monitor error types in HTML Dashboard; reduce ramp-up aggression if rate-limiting errors appear.
DemoBlaze API endpoints may change without notice (third party application).	**Medium**	**Medium**	Validate all seven endpoints manually in a browser before each test execution session.
JMeter OutOfMemoryError during 500-user stress test.	**Medium**	**High**	Increase JVM heap to -Xmx4g in jmeter.bat before running stress test.
Test data (registered user accounts) may be cleared if DemoBlaze resets its demo database.	**Low**	**Medium**	Re-register test accounts before each testing session.
Test machine becomes a performance bottleneck at high thread counts (300–500 users).	**Medium**	**Medium**	Allocate at least 4 GB of RAM to JMeter; monitor CPU and memory usage during stress runs.
Old .jtl results file or reports folder causing JMeter startup error.	**High**	**Low**	Always delete .jtl files and report folders before each run.

CI/CD runner network restrictions blocking demoblaze.com.	**Low**	**Medium**	Verify connectivity in pipeline; fallback to local execution if blocked.

2. Responsibilities

This section defines who is responsible for each aspect of the performance testing effort. Clear ownership prevents tasks from being missed and ensures accountability throughout the test cycle.

Role	Responsibility	Owner
QA Engineer	Design and maintain the JMeter test plan; execute all test phases; generate and review HTML reports; document findings in the final report.	QA Team
QA Lead / Reviewer	Review and approve the test plan before execution; review final report findings and recommendations.	QA Lead
DevOps / CI Engineer	Configure GitHub Actions pipeline, manage secrets, review pipeline output.	DevOps Team
Developer (Backend)	Provide clarification on application architecture where needed.	Dev Team

3. Performance Goals & Acceptance Criteria

Performance goals translate the project requirements into concrete, measurable targets. Without defined thresholds, it is impossible to declare a test as passed or failed. Each metric below has three bands — target, warning, and critical — which will be used as the basis for assertions in JMeter and for pass/fail decisions in the CI/CD pipeline.

3.1 Key Performance Indicators (KPIs)

Metric	Target (Pass)	Warning Threshold	Critical (Fail)
Average Response Time	< 1.5 seconds	1.5 – 2.0 seconds	> 2.0 seconds
90th Percentile (P90)	< 2.0 seconds	2.0 – 2.5 seconds	> 2.5 seconds
95th Percentile (P95)	< 2.5 seconds	2.5 – 3.5 seconds	> 4.0 seconds
Throughput	>= 500 req/sec	400 – 499 req/sec	< 400 req/sec
Error Rate	< 0.5%	0.5% – 1.0%	> 1.0%
Peak Concurrent Users	300 users stable	N/A	N/A
Stress Breaking Point	> 400 users before failure	N/A	< 300 users
Endurance Degradation	< 10% increase over 30 min	10–20% increase	> 20% increase

Entry and Exit Criteria

Entry Criteria

Testing will commence only when all of the following conditions are met:

- JMeter 5.6.3 is installed and the --version flag returns the correct version.
- The JMX test plan has been validated (XML parses without error).
- Internet connectivity to www.demoblaze.com is confirmed — site loads successfully in browser.
- The results and reports directories exist and are empty.
- The GitHub repository contains the latest JMX file and pipeline YAML.
- All CSV test data files are populated and verified.

Exit Criteria

Testing will be considered complete when all of the following conditions are met:

- All five test phases have been executed and produced valid .jtl results files.

- HTML Dashboard reports have been generated for all five test phases.
- The GitHub Actions pipeline has been triggered and all jobs have completed successfully.
- All identified performance findings have been documented in the final report.
- The final performance report has been reviewed and submitted.

Test Types

Five distinct types of performance tests will be conducted, each serving a specific purpose in evaluating a different aspect of the application's performance characteristics.

- The Baseline Test will be run with 50 concurrent users over approximately 5 minutes. Its purpose is to establish a reference point for normal application performance under minimal load. All subsequent test results will be compared against this baseline to identify performance changes under higher loads.
-
- The Load Test will simulate the expected normal daily traffic using 150 concurrent users over 15 minutes. This test will validate that the application performs acceptably under typical usage conditions and that response times and error rates remain within the defined acceptance criteria.
-
- The Peak Load Test will push the application to 300 concurrent users over 10 minutes, simulating maximum expected traffic during a high-demand event such as a product launch or promotional campaign. The goal is to confirm that the application can sustain peak demand without performance degradation.
-
- The Stress Test will deliberately exceed the system's designed capacity by simulating 500 or more concurrent users. The objective of stress testing is not to pass the performance thresholds, but to identify the breaking point — the user load level at which the application begins to fail or degrade significantly.

The Endurance Test will sustain 150 concurrent users for a period of 30 minutes. Unlike the other test types, endurance testing focuses on detecting gradual performance degradation over time, such as

memory leaks, connection pool exhaustion, or database query slowdowns that only emerge during prolonged operation.

4.2 Test Data Strategy

Test data management is one of the most important aspects of performance testing. Poorly managed test data can produce misleading results. This section describes how data will be prepared and fed into the JMeter test plan to ensure consistent and repeatable execution.

User credentials will be stored in a CSV file (test_users.csv) containing pre-registered DemoBlaze test accounts. The CSV Data Set Config element in JMeter will feed credentials to virtual user threads during test execution. Order data for the checkout transaction will be parameterised using a second CSV file (order_data.csv) containing checkout details including name, country, city, card number, and expiry date. Product IDs used in the View Product Detail and Add to Cart requests will be fixed to known stable products in the DemoBlaze catalogue, verified manually before test execution begins.

5. Test Scenarios and Transactions

Each virtual user session follows the transaction sequence defined below. Think times between requests use Gaussian Random Timers to simulate realistic user behaviour. All transactions include HTTP 200 response code assertions. The JMeter test plan uses a single parameterised thread group controlled by command-line properties (-Jthreads, -Jrampup, -Jloops), allowing one JMX file to serve all five test phases without modification.

5.1 Transaction Flow

Step	Transaction Name	Method	URL Path	Think Time	Assertion
1	Load Home Page	**GET**	/index.html	2000ms ± 1000ms	**HTTP 200**
2	View Product Detail (Samsung S6)	**GET**	/prod.html?idp_=1	2000ms ± 1000ms	**HTTP 200**

3	Browse Another Product (Nokia)	**GET**	/prod.html?idp_=2	2000ms ± 1000ms	**HTTP 200**
4	View Cart Page	**GET**	/cart.html	1500ms ± 500ms	**HTTP 200**
5	Reload Home Page	**GET**	/index.html	1000ms ± 500ms	**HTTP 200**
6	View Cart Again	**GET**	/cart.html	1000ms ± 500ms	**HTTP 200**

5.2 Thread Group Configuration

The following table defines the thread group configuration for each test phase:

Test Phase	TG	Users	Ramp-Up	Loops	Duration	Purpose
Baseline	TG1	50	60s	1	~5 min	Establish performance baseline
Load	TG2	150	120s	1	~8 min	Simulate normal production load
Peak	TG3	300	180s	2	~12 min	Simulate peak traffic conditions
Stress	TG4	500	120s	3	~5 min	Identify application breaking point
Endurance	TG5	100	120s	20	~30 min	Detect sustained load degradation

5.3 Think Time Configuration

Gaussian Random Timers are placed between each HTTP request to simulate realistic user behaviour and prevent artificial request bursts:

- After Home Page load: 2000ms ± 1000ms
- After Product Detail views: 2000ms ± 1000ms

- After Cart Page load: 1500ms ± 500ms
- After subsequent pages: 1000ms ± 500ms

6. JMeter Test Plan Configuration

This section documents the technical setup of the JMeter test plan (.jmx file). It ensures the test plan is fully transparent and reproducible. Anyone opening the JMX file should be able to understand why each element is present and how it is configured by referencing this section.

6.1 Test Plan Structure

The JMeter test plan will be saved as a single .jmx file containing all five thread groups. Only one thread group will be enabled at a time during execution — the remaining four will be disabled in JMeter GUI before each run. This approach keeps the entire test suite in one maintainable file while allowing clean, isolated execution of each test phase.

The following global configuration elements will be defined at the test plan level and shared across all thread groups:

- HTTP Request Defaults — specifying the base domain (www.demoblaze.com), protocol (HTTPS), connection timeout (10 seconds), and response timeout (30 seconds).
- HTTP Cookie Manager — configured to clear cookies on each iteration to simulate fresh user sessions.
- HTTP Header Manager — setting the Content-Type, Accept, User-Agent, and Origin headers required by the DemoBlaze API.

6.2 Assertions

Assertions act as automated pass/fail checks on each response. Without assertions, JMeter counts every response as a success, even if the server returns an error message. Assertions are what give the error rate metric its meaning.

Two types of assertions will be applied to every sampler in the test plan:

- **Response Code Assertion** — verifies that each HTTP response returns status code 200 (OK). Any response with a 4xx or 5xx status code will be marked as a failed sample and counted in the error rate.
- **Duration Assertion** — verifies that each response is received within 2000 milliseconds. Any response that exceeds this threshold will be marked as a performance failure, even if it returned a valid 200 status code.
- **CI/CD Threshold Assertions** — the GitHub Actions pipeline includes a Python-based post-execution threshold check that parses the .jtl results file and fails the pipeline if the error rate exceeds 1%, providing automated quality gates on every code push.

6.3 Listeners and Reporting

Listeners are JMeter's data collection components. They capture raw test data and present it in usable formats. During load runs, only lightweight listeners will be enabled to avoid consuming excessive memory.

Output Type	Generated By	Location	Purpose
JTL Results File	JMeter -l flag	C:\swaglab\results\<test>.jtl | CI: GitHub Actions Artifacts	Raw CSV data for analysis and report regeneration
HTML Dashboard Report	JMeter -e -o flags	C:\swaglab\reports\<test>\index.html	Interactive visual dashboard with all performance charts
GitHub Actions Artifact	upload-artifact step	Actions run → Artifacts section	Downloadable HTML report per CI/CD run
GitHub Step Summary	Python summary step	Actions run → Summary tab	Consolidated results table in pipeline run view

The View Results Tree listener will be included but disabled by default — it should only be enabled during single-user debugging runs in JMeter GUI, as it stores full response bodies and will consume large amounts of memory during load tests.

6.4 HTML Dashboard Contents

The JMeter HTML Dashboard (generated with the -e -o flags) provides the following visualisations for each test run:

- Statistics table — sample count, average, min, max, P90, P95, P99, error rate, throughput
- Response Times Over Time chart — trends across the test duration
- Active Threads Over Time chart — ramp-up and ramp-down visualisation
- Bytes Throughput Over Time — data transfer rates
- Response Time Distribution — histogram of response time spread
- Response Time Percentiles — P50 through P99 visualisation
- Top 5 Errors by Sampler table — identifies the highest-failure transactions

7. Test Execution Plan

The test execution plan specifies the exact sequence in which tests will be run, the conditions that must be satisfied before testing begins, and the actions to be taken after each run. A structured execution plan prevents common mistakes such as running tests with stale data, using incorrect thread group configurations, or failing to collect results properly.

7.1 Execution Sequence

Tests will be executed in the following order. Each phase must be completed and reviewed before proceeding to the next. Running tests in order from lowest to highest load ensures the baseline is clean and that any issues identified at lower loads are understood before stress testing begins.

#	Test Phase	Users	Ramp-Up	Duration	Output File
1	Baseline Test	50	50 seconds	~5 minutes	baseline.jtl
2	Load Test	150	120 seconds	~15 minutes	load.jtl
3	Stress Test	300	180 seconds	~10 minutes	stress.jtl

4	Peak Stress Test	500	250 seconds	~10 minutes	peak.jtl
5	Endurance Test	150	120 seconds	30 minutes	endurance.jtl

7.2 Pre-Execution Checklist

The following checklist must be completed before starting any test run. Do not begin execution if any item cannot be confirmed.

- Confirm internet connectivity to www.demoblaze.com (open in browser and verify load)
- Confirm JMeter 5.6.3 is installed and the --version flag returns the correct version
- Confirm JMX test plan is present and XML parses without error
- Confirm results folder is empty — delete any previous .jtl files
- Confirm reports folder is empty or deleted (JMeter throws an error if the output folder already exists)
- Confirm CSV test data files (test_users.csv, order_data.csv) are present and populated
- Confirm JMeter heap is set to at least -Xmx1g in jmeter.bat (or -Xmx4g for stress tests)
- Confirm the correct thread group is enabled and all others are disabled in the JMX file
- Confirm the GitHub repository contains the latest JMX file and pipeline YAML (for CI runs)

7.3 Test Execution Steps

Each test run will be executed using the following procedure. Open a command prompt and navigate to the JMeter bin directory. Execute the JMeter command in non-GUI mode specifying the test plan file (-t), the results output file (-l), and the HTML report output directory (-e -o):

```
jmeter.bat -n -t "test.jmx" -Jthreads=<N> -Jrampup=<R> -Jloops=<L> -l
"results/<test>.jtl" -e -o "reports/<test>"
```

Monitor the command prompt for summary lines that appear every 30 seconds showing the current request count, average response time, and error rate. Allow the test to run to completion. Once the test ends and the END OF TEST message appears, verify that the .jtl file has been created and that the HTML report folder contains an index.html file.

7.4 Post-Execution Steps

After each test run completes, the following steps will be performed:

- Open the HTML Dashboard report (html-reports/[testname]/index.html) in a browser and review key visualisations including response time percentiles, throughput over time, and error rates.
- Record the overall average response time, 90th percentile response time, throughput, and error rate for inclusion in the final performance report.
- Document any assertion failures, unexpected error types, or anomalous response time spikes observed during the run.
- Save a copy of the .jtl results file — it may be needed to regenerate the HTML report or perform additional analysis.
- Once the current phase has been reviewed and documented, enable the next thread group in JMeter, disable the current one, save the file, and proceed to the next test run.

7.5 Suspension and Resumption Criteria

Testing will be suspended if any of the following conditions occur:

- The DemoBlaze application becomes completely unresponsive and returns no responses for more than 2 minutes.
- The error rate across all transactions exceeds 50% of requests, suggesting a fundamental environment issue rather than a performance issue.
- The test machine runs out of memory and JMeter crashes.
- A network outage prevents requests from reaching the application.

Testing may resume once the cause of the suspension has been identified and resolved, the application has been verified as accessible and functional, and the test data and results folders have been cleaned up to ensure the next run starts from a fresh state.

8. Metrics and Reporting

This section defines what will be measured, how it will be measured, and what format the results will be delivered in. Defining metrics before testing begins prevents selective reporting of only favourable results and ensures all stakeholders know exactly what to expect in the final deliverable.

8.1 Metrics to be Collected

The following metrics will be collected and analysed from each test run:

- **Average Response Time** — mean time from request sent to full response received, measured in milliseconds. The primary indicator of user-perceived performance.
- **90th Percentile (P90)** — 90% of all requests were served within this time. Provides a better view of typical user experience than the average as it is less affected by outliers.
- **95th Percentile (P95)** — provides a stricter view of worst-case performance for the majority of users.
- **Throughput** — number of requests completed per second; indicates the application's capacity under load.
- **Error Rate** — percentage of total requests that resulted in a failure (non-200 HTTP status code or duration assertion violation).
- **Minimum and Maximum Response Times** — show the fastest and slowest individual request times.
- **Active Threads Over Time** — concurrent user count at each point during the test, useful for correlating performance changes with load increases.
- **Latency** — time until the first byte of the response is received, helping to distinguish network delays from server processing time.
- **Endurance Degradation Rate** — percentage increase in average response time between the beginning and end of the endurance test window, used to detect gradual performance degradation.

8.2 HTML Dashboard Reports

An HTML Dashboard report will be generated for each of the five test runs — Baseline, Load, Peak, Stress, and Endurance. Each report will be generated automatically by JMeter at the end of the command-line test run and will include: a Response Time Percentiles graph; a Throughput Over Time graph; an Error Rate Over Time graph; an Active Threads Over Time graph; a per-transaction Statistics table; and a Top 5 Errors by Sampler table.

9. Test Deliverables

The following deliverables will be produced as part of this performance testing engagement. Status will be updated upon completion of each item.

Deliverable	Description	Status
Performance Test Plan (.docx)	The primary planning and reference document.	**Complete**
JMeter Test Plan (.jmx file)	Executable test plan containing all thread groups, samplers, assertions, timers, and listeners.	**Complete**
HTML Dashboard — Baseline	50 users, 3 loops, ~5 minutes	**Complete**
HTML Dashboard — Load	150 users, 3 loops, ~8 minutes	**Complete**
HTML Dashboard — Peak	300 users, 2 loops, ~12 minutes	**Complete**
HTML Dashboard — Stress	500 users, 1 loop, ~5 minutes	**Complete**
HTML Dashboard — Endurance	100 users, 20 loops, ~30 minutes	**Complete**
CI/CD Pipeline (.yml)	GitHub Actions pipeline with automated email and Slack notifications.	**Complete**
Final Performance Report (.docx)	Summary document presenting all findings, KPI comparisons, root cause analysis, and recommendations.	**Complete**

10. CI/CD Integration

CI/CD integration embeds performance tests into the software development pipeline so that performance regressions are detected automatically on every code change, rather than only before releases. This section describes the planned GitHub Actions pipeline that will automate JMeter test execution.

10.1 Overview

Performance tests will be automated within a CI/CD pipeline using GitHub Actions. The pipeline will be triggered on pull requests to the main branch, on a nightly scheduled run, and on manual trigger via the workflow_dispatch event. The pipeline will automatically download and configure JMeter, execute the

test plan, generate the HTML Dashboard report, upload the report as a build artefact, and fail the build if the defined performance thresholds are breached.

10.2 Pipeline Trigger Strategy

Trigger	Tests to Run	Rationale
Pull Request to main branch	Baseline Test (50 users)	Fast feedback on regressions — completes in ~5 minutes
Nightly schedule (1AM UTC)	Baseline + Load + Stress	Full nightly regression check
Manual trigger (workflow_dispatch)	User-selectable test type	On-demand execution for stress or endurance runs

10.3 Pipeline Jobs

Job Name	Users	Depends On	Artifacts Uploaded
baseline-test	50	None	baseline-html-report, baseline-jtl
load-test	150	baseline-test	load-html-report, load-jtl
stress-test	500	load-test	stress-html-report, stress-jtl
summary	N/A	All jobs	GitHub Step Summary report

10.4 Automated Pass/Fail Gates

The CI/CD pipeline will enforce the following automated quality gates. A pipeline run will be marked as FAILED and the corresponding pull request will be blocked from merging if:

- The average response time exceeds 2000 milliseconds for any critical transaction.
- The overall error rate (excluding known endpoint issues) exceeds 1% of total requests.

These gates ensure that no code change which degrades application performance beyond acceptable thresholds can be merged without review.

10.5 Notifications

The pipeline is configured to send automated notifications on completion of each test run:

- **Email Notification** — sent via Gmail SMTP to the configured recipient with a full HTML report including pass/fail status, sample counts, error rate, average response time, and links to GitHub Actions artifacts. Configured via GMAIL_ADDRESS, GMAIL_APP_PASS, and REPORT_EMAIL secrets.
- **Slack Notification** — sent via Slack Incoming Webhook with a structured message block including test summary metrics, run details, and direct links to the GitHub Actions run and artifact download. Configured via SLACK_WEBHOOK_URL secret.

11.2 Assumptions

The following assumptions have been made in preparing this test plan:

- The DemoBlaze application will be accessible and functionally stable during test execution.
- Network latency from the test machine to the DemoBlaze servers will remain reasonably consistent throughout each test run.
- Apache JMeter version 5.6.3 will be used for all test executions.
- Test user accounts registered on DemoBlaze will remain active for the duration of the testing engagement.
- The test machine will have a minimum of 4 GB of available RAM and a stable internet connection throughout testing.

11.3 Constraints

This test plan operates under the following constraints that cannot be changed. DemoBlaze is a publicly hosted demo application — there is no access to server-side infrastructure, logs, or monitoring tools such as Grafana or Prometheus. Performance results will therefore reflect the combined effect of application performance and network latency from the test machine to DemoBlaze's hosting environment. Server-side metrics such as CPU, memory, and database connection pool usage cannot be directly observed or reported.

12. Test Environment

The test environment section documents the hardware, software, network, and tool configuration that will be in place when testing is executed. Documenting the environment is essential for reproducing test results and for identifying whether any environmental factors may have influenced the outcome.

12.1 Environment Overview

Application Under Test	DemoBlaze — https://www.demoblaze.com
Protocol	HTTPS (TLS)
Performance Testing Tool	Apache JMeter 5.6.3
JVM Configuration	OpenJDK 11, -Xms1g -Xmx4g
Load Generator OS	Windows 10/11 (local runs) / Ubuntu 22.04 (GitHub Actions runs)
CI/CD Platform	GitHub Actions — ubuntu-latest runner
Repository	github.com/[your-org]/demoblaze-performance-tests
Results Storage	Local: C:\swaglab\results\ | CI: GitHub Actions Artifacts
Report Format	JMeter HTML Dashboard (auto-generated with -e -o flags)

12.2 Test Machine Specifications

Performance tests will be executed from a single test machine. The machine should meet the following minimum specifications to support high thread counts without becoming a performance bottleneck itself:

Component	Minimum Requirement
Operating System	Windows 10 (64-bit) or later / Ubuntu 22.04 (CI runner)
Processor	Intel Core i5 or equivalent — 2.0 GHz or higher
RAM	Minimum 4 GB available for JMeter heap allocation
Storage	Minimum 2 GB free disk space for results and reports
Network	Stable internet connection — minimum 10 Mbps
Java Runtime	JDK 11 or higher (required by JMeter 5.6.3)

12.3 Performance Testing Tool

Apache JMeter version 5.6.3 will be used as the primary performance testing tool. JMeter will be run in non-GUI (command-line) mode for all load test executions to conserve system resources and maximise the number of supported threads. The JMeter GUI will be used only for test plan configuration, debugging single-user runs, and viewing reports.

The JMeter heap memory allocation will be configured to a minimum of 1 GB (-Xms1g -Xmx1g) in the jmeter.bat file for baseline and load tests, and a minimum of 4 GB (-Xms1g -Xmx4g) for stress and peak tests to support high thread counts.

12.4 Application Environment

The application under test, DemoBlaze Product Store, is hosted in a public cloud environment at https://www.demoblaze.com. This is a shared demo environment accessible to the general public. No dedicated test environment is available for this assessment, and no server-side access will be granted to the testing team. Test results may be affected by concurrent usage from other users during test execution.

13. Testing Resources

This section documents the human resources and tools required to execute this performance test plan.

13.1 Human Resources

Role	Responsibilities
QA Engineer / Scrum Master	Designs test plan, configures JMeter test plan, executes all test phases, analyses results, prepares final report.
DevOps / CI Engineer	Configures GitHub Actions pipeline, manages secrets, reviews pipeline output.
Developer (Backend)	Provides clarification on application architecture where needed.
Product Owner / Instructor	Reviews test plan and final report, provides sign-off.

13.2 Tools and Software

Tool	Version	Purpose
Apache JMeter	5.6.3	Primary load generation and test execution tool
OpenJDK	11	JVM runtime for JMeter
GitHub Actions	N/A	CI/CD pipeline automation
Git	2.46.0	Version control and repository management
Python	3.x	Results parsing and threshold checking in CI pipeline
Gmail SMTP	N/A	Email notification delivery
Slack Incoming Webhooks	N/A	Slack notification delivery

15. Test Schedule

The test schedule provides an estimated timeline for all performance testing activities. This allows stakeholders to plan around testing activities and ensures all deliverables are completed before the submission deadline.

Activity	Estimated Duration	Description
Test Plan Review and Approval	1 day	Stakeholder review and sign-off on this document
JMeter Test Plan Setup	1 day	Configure test plan, validate all endpoints, prepare CSV data files
Baseline Test Execution	~1 hour	Run baseline, review HTML report, document findings
Load Test Execution	~1 hour	Run load test, review HTML report, document findings
Stress Test Execution	~1 hour	Run stress test, review HTML report, document findings

Endurance Test Execution	~2 hours	Run 30-minute endurance test, review HTML report
CI/CD Pipeline Setup	~2 hours	Configure and validate GitHub Actions workflow
Final Performance Report	1 day	Compile all findings, analysis, and recommendations

Communication Plan

The following defines how testing information will be shared across the team throughout the performance testing project.

Regular Meetings: Daily stand-ups are held every morning via Microsoft Teams to discuss test progress, blockers, and next steps. Sprint review meetings are held at the end of each sprint where the QA Engineer presents the HTML Dashboard reports, JMeter test plan walkthrough, and performance findings to all stakeholders.

Status Reports: Weekly status reports summarizing test results, error rates, and SLA compliance are shared with the Product Owner and team via email.

Instant Messaging: Slack and Microsoft Teams are used for rapid day-to-day communication between testers, developers, and DevOps for quick updates and blocker escalation.

Automated Notifications: The GitHub Actions pipeline is configured to send automated email and Slack notifications after every test run. Each notification includes build status, error rate, average response time, pass rate, and direct links to the HTML Dashboard artifacts.

Defect Communication: Any performance finding that breaches the defined SLAs — error rate above 1% or response time above 2000ms — is immediately logged in Jira and escalated to the backend developer and Product Owner within the same day.

Deliverables Sharing: All HTML Dashboard reports, the JMeter test plan (.jmx), and the final performance report (.docx) are shared via GitHub Actions artifacts and email at the end of each test phase.